PROJECT OVERVIEW

slideotter

A local workbench for structured presentations that stay grounded, reviewable, and easy to publish.

This tour shows new users where the work lives and how an idea becomes an archived PDF.

Structured source

Slides, context, sources, and materials stay inspectable in the project tree.

Live workbench

Preview, edit, attach images, compare variants, and plan decks in one browser flow.

Guarded output

Generated changes wait for review, then validation protects the archive.

TOUR MAP

What you will learn

Choose a deck, ground it, review changes, and ship a checked archive.

Start

Pick or create a deck with constraints and sources.

Author

Edit JSON, attach materials, keep context close.

Review and ship

Compare candidates, run checks, and archive the intended version.

The flow works for the demo or a new deck.

PROBLEM

Why decks get messy

Deck work gets hard when source, preview, and final PDF stop agreeing.

What usually drifts

Source
The file no longer explains the visible slide.

Context
Audience and tone live outside the deck.

Options
Ideas replace originals before review.

Output
PDF issues appear after export.

What slideotter keeps together

One local loop
Edit, preview, compare, and validate in the studio.

Visible source
Keep JSON close to the rendered slide.

Intentional archive
Validate before archiving.

WORKFLOW

A simple authoring loop

Most work follows four steps and leaves readable local state.

Daily flow

1. **Plan**
   Capture objective, outline, audience.
2. **Edit**
   Change text or JSON beside preview.
3. **Review**
   Compare alternatives before applying.
4. **Ship**
   Build, validate, archive.

Why it stays calm

Local
Runs against this repository.

Explicit
Buttons request bounded actions.

Inspectable
State stays readable on disk.

WORKSPACE

Choose a presentation first

slideotter manages multiple local decks with visual first-slide cards.

What a deck owns

Slides
Specs under presentations/<id>/slides.

Context
Brief, outline, theme, checks.

Sources
Notes, excerpts, URLs.

Materials
Images, credits, metadata.

What you can do there

- **Create**
  Set brief, length, sources.
- **Duplicate**
  Copy a deck when you want a branch.
- **Delete**
  Remove a deck only after confirmation.

CONTEXT

Give the deck a brief and sources

Before generation, state the goal and trusted material.

Useful brief fields

Audience
Who needs this?

Objective
What should change?

Target length
First-draft slide count.

Starter sources
Notes, excerpts, URLs.

Where the brief helps

Planning
Uses brief and outline.

Grounding
Uses bounded snippets.

Scaling
Skip, restore, add detail.

SOURCE MODEL

Slides are readable JSON

Each structured slide is a compact JSON file. The renderer turns those fields into the slide you inspect in the studio.

Current slide families

Cover
Identity, summary, logo, cards.

TOC
Tour framing and section cards.

Content
Evidence and guardrail panels.

Summary
Takeaways and resource cards.

Why JSON helps

Parseable
Invalid JSON cannot be saved.

Diffable
Changes stay reviewable in source.

Extensible
New families can be added later.

SLIDE STUDIO

Edit with the slide in view

The main preview is the working surface. Text and JSON edits update it immediately.

Edit path

1. **Select**
   Choose the target in the slide strip.
2. **Inline**
   Double-click editable preview text.
3. **JSON**
   Open the spec drawer.
4. **Save**
   Persist valid changes.

What prevents mistakes

Parse errors block save
Invalid JSON cannot persist.

Selected slide only
Actions stay scoped.

Preview stays central
Editing chrome is secondary.

MEDIA

Attach images as materials

Upload, search, or provide starter images. Materials stay with the deck and carry source information.

Material flow

Upload
Files land in presentations/<id>/materials.

Search
Openverse and Wikimedia can import images.

Credit
Provider, creator, license, and source are saved.

Attach
Generation or the author can attach one image.

Practical rules

- **Alt text**
  Make the image understandable.
- **Source line**
  Captions keep visible credit close.
- **Explicit attach**
  Upload alone never edits a slide.

RUNTIME

One renderer keeps previews honest

The studio avoids separate preview and export logic. The same DOM slide powers authoring, comparison, validation, and PDF output.

Surfaces using it

Main preview
The large slide in Slide Studio.

Thumbnails
Selector and presentation cards.

Compare panes
Current and candidate views.

PDF export
Playwright prints the DOM output.

Why it matters

One path
Fewer places for layout drift.

Browser measured
Checks inspect rendered geometry.

Deck themed
Slides keep their own visual theme.

VARIANTS

Generate options, not surprises

Generation creates review candidates, reports status, and keeps the current slide unchanged until Apply.

What you control

How many

Choose candidate count; five is default.

What kind

Try wording, structure, layout, or theme.

Provider

Use local rules, OpenAI, LM Studio, or OpenRouter.

Progress

LLM substatus shows requests and streamed chunks.

Apply discipline

Session-only

Candidates stay outside slide files until applied.

Grounded

Briefs, sources, and materials feed prompts.

Shortened

Overlong text can be rewritten semantically.

REVIEW

Compare before applying

When candidates exist, the compare workspace becomes the decision surface: current slide, candidate slide, changed fields, and apply buttons.

What to inspect

Current

The slide you would replace.

Candidate

The proposed version.

Changes

Touched fields and line diffs.

Apply

Promote with optional validation.

Decision cues

Hidden until useful

No empty comparison area.

Direct selection

Rows choose the candidate to inspect.

No surprise writes

Nothing changes without Apply.

DECK PLANNING

Plan the deck above slide level

Deck Planning handles outline, sources, length, and shared context.

Plan candidate contents

1. **Sources**
   Add notes, excerpts, URLs.
2. **Scale length**
   Skip, restore, or add detail.
3. **Context**
   Brief, theme, or constraints patch.
4. **Preview**
   Current and proposed strips.

Promotion rules

Skip, do not delete
Skipped slides can return.

Paths are named
Review touched files.

Apply once
Plan promotion is explicit.

ASSISTANT

Use chat for bounded tasks

In Slide Studio, chat launches bounded workflows and keeps history in a Log tab.

Inputs it sees

Message
The request you type.

Selection
Text from the preview.

Slide
Current slide id and context.

Log
Workflow events stay separate.

How it stays inspectable

Known actions
Triggers studio workflows.

Studio only
Hidden while browsing decks.

Provider popover
LLM details live in nav.

SAFETY

Know what can change

The server keeps writes narrow. Browser actions request specific mutations instead of giving the UI broad filesystem access.

Allowed targets

Slide specs
Structured JSON in the active presentation.

Presentation state
Context, variants, materials, and preferences.

Materials
Image files for the active deck.

Generated output
Preview pages, PDFs, and archive copies.

Change discipline

- **Allowlisted paths**
  Unexpected targets are rejected.
- **Named actions**
  Add, remove, attach, compare, apply.
- **Separate archive**
  Sharing output is deliberate.

VALIDATION

Run checks before sharing

Validation stays subtle in the UI, but it is the final safety net before a PDF becomes the shared version.

Gate coverage

Types
Studio, server, and scripts compile.

Layout
Rendered slide bounds, text, and contrast.

Media
Images, captions, sources, and spacing.

Browser
Studio workflows and page chrome.

What failures mean

Baseline drift
Refresh only intentional visual changes.

Attribution
Source lines stay attached to images.

Fixtures
Known regressions stay covered.

OUTPUT

Archive the version you mean to share

Building and archiving are separate. Build creates current output; archive update copies the presentation PDF only when you intend to publish it.

Artifact path

1. **Build**
   slides/output/<presentation-id>.pdf.
2. **Render pages**
   studio/output/<id>/rendered-pages.
3. **Baseline**
   studio/baseline/<id>.
4. **Archive**
   archive/<presentation-id>.pdf.

Routine

Regenerate
Build after slide or theme changes.

Validate
Run the quality gate.

Archive
Update the checked-in snapshot.

FOR CONTRIBUTORS

How to read the project

If you are new to the codebase, start with the folder that matches the change.

Where to look

presentations/
Deck source, context, materials, state.

studio/
Browser app, server, renderer, baselines.

scripts/
Build, validation, archive commands.

docs/ and status
ADRs, roadmap, status notes.

Working habits

Small changes
Keep edits close to the boundary.

Docs follow code
Update guidance when workflows move.

Gate locally
Run quality checks before committing.

ROADMAP

Good next improvements

The useful path is smoother grounded deck work, not a bigger generic editor.

Likely next slices

Grounding quality
Tune retrieval against real decks.

Citation discipline
Decide when source-backed claims surface.

Appification
Make slideotter runnable from any workspace.

Onboarding
Keep examples, screenshots, and archive current.

Scope to avoid

- **No full WYSIWYG chase**
  Structured decks are the point.
- **No second renderer**
  The DOM path remains authoritative.
- **No blind apply**
  Generated changes stay reviewable.

TAKEAWAY

The basic habit

Keep the story, sources, materials, preview, review, validation, and archive in one local loop.

- **Start with intent**

  Choose a presentation, write the brief, add sources, then edit the slides that serve it.

- **Review every option**

  Use variants, deck plans, and semantic scaling as proposals.

- **Ship only checked output**

  Build, validate, refresh baselines when needed, and update the archive.

DAILY COMMANDS

Build

npm run build

Quality gate

npm run quality:gate